

Rapport DevOps — Mini Helpdesk API

Module: DevOps | Année: 2025-2026 | Sujet: Plateforme de tickets de support (mini Helpdesk).

Étudiant 1: Mouaad Chakir

Étudiant 2: Binome

Dépôt GitHub: <https://github.com/mouaadchakir/devops-helpdesk-api>

Ce rapport présente la conception, la réalisation et l'industrialisation d'une API REST de gestion de tickets, incluant Git workflow, Docker, CI/CD et bonus Kubernetes.

1. Introduction

Ce projet a pour objectif de mettre en œuvre une chaîne DevOps complète autour d'une application métier. Le sujet retenu est un mini Helpdesk où des utilisateurs créent des tickets, des agents traitent ces tickets et des administrateurs supervisent le flux.

Les objectifs techniques étaient: développer une API robuste, assurer la persistance des données, sécuriser les accès par rôles, conteneuriser l'application, automatiser les tests/builds et préparer un déploiement Kubernetes en bonus.

1.1 Périmètre fonctionnel

- Création/consultation de tickets
- Affectation de tickets à un agent
- Changement de statut: OPEN, IN_PROGRESS, RESOLVED, CLOSED
- Ajout de réponses agent sur un ticket
- Authentification et autorisations par rôles

Le choix d'une API REST documentée répond explicitement à l'énoncé qui autorise "interface utilisateur ou API REST documentée".

2. Architecture générale

2.1 Vue globale

L'architecture repose sur Spring Boot 3.3.x avec couche Web, Service et Repository. La base MySQL stocke les utilisateurs, rôles, tickets et réponses. Spring Security gère l'authentification HTTP Basic et les règles d'autorisation selon le rôle.

2.2 Schéma logique

- Client REST (Swagger/curl/Postman)
- API Spring Boot (Controllers)
- Services métier (règles de gestion)
- Repositories JPA (accès données)
- MySQL (persistance)
- CI/CD GitHub Actions + Registre GHCR

Cette séparation simplifie la maintenance, la testabilité et l'évolutivité du projet.

2.3 Modèle de données

- AppUser: username, email, password, rôles
- Role: ROLE_USER, ROLE_AGENT, ROLE_ADMIN
- Ticket: titre, description, priorité, statut, créateur, agent assigné
- TicketResponse: message, auteur, ticket, date de création

3. Implémentation applicative

3.1 Endpoints exposés

- POST /api/auth/register
- GET /api/auth/me
- POST /api/tickets
- GET /api/tickets
- GET /api/tickets/{id}
- PATCH /api/tickets/{id}/assign
- PATCH /api/tickets/{id}/status
- POST /api/tickets/{id}/responses

3.2 Règles de sécurité

Les endpoints d'affectation, changement de statut et réponses sont réservés aux rôles AGENT/ADMIN. L'accès liste/détail ticket est filtré selon le rôle: ADMIN voit tout, AGENT voit ses tickets assignés, USER voit ses tickets créés.

3.3 Initialisation des données

Au démarrage, l'application crée les rôles et trois comptes par défaut (user/agent/admin) afin de simplifier les tests et démonstrations.

4. Gestion de versions (Git)

Le dépôt est structuré autour de deux branches principales: develop (intégration) et main (release). Les évolutions sont prévues via feature/* + Pull Requests.

4.1 Bonnes pratiques appliquées

- Commits atomiques avec messages explicites
- Historique lisible (setup, domain model, API, Docker, CI/CD, docs)
- Template de Pull Request ajouté dans .github/pull_request_template.md
- CODEOWNERS configuré pour renforcer la relecture

4.2 Intérêt DevOps

Ce workflow permet de contrôler la qualité avant intégration, de tracer les changements et de sécuriser les releases.

5. Conteneurisation Docker

L'application est conteneurisée avec un Dockerfile multi-stage. La phase builder compile le JAR avec Maven, puis une image runtime légère exécute le jar final.

5.1 Fichiers fournis

- Dockerfile (multi-stage, Java 17)
- docker-compose.yml (MySQL)
- .dockerignore

5.2 Commandes clés

- `docker build -t helpdesk-api:local .`
- `docker compose up -d`

Ce choix garantit une exécution reproductible et facilite le déploiement sur différents environnements.

6. Pipeline CI/CD

Le workflow GitHub Actions (ci-cd.yml) automatise la qualité et la livraison.

6.1 Étapes automatisées

- Checkout du code
- Setup Java 17 + cache Maven
- Tests Maven
- Build package
- Build image Docker
- Push image GHCR sur push main

6.2 Politique de publication

Les builds se déclenchent sur push/pull_request de main et develop. La publication du conteneur est limitée à main pour maintenir un artefact stable de production.

7. Bonus Kubernetes

Un dossier k8s/ a été ajouté pour démontrer un déploiement container orchestré.

7.1 Manifestes fournis

- configmap.yaml (variables Spring)
- secret.yaml (credentials datasource)
- mysql-deployment.yaml + mysql-service.yaml
- app-deployment.yaml + app-service.yaml
- kustomization.yaml

7.2 Déploiement

- kubectl apply -k k8s/
- kubectl get pods,svc

Le service applicatif est exposé en NodePort (30080) pour les tests locaux sur Minikube/cluster équivalent.

8. Tests, qualité et observabilité

Les tests de base Spring Boot sont exécutés dans le pipeline pour valider le démarrage du contexte applicatif et la cohérence des dépendances.

La documentation OpenAPI (Swagger UI) facilite la validation fonctionnelle des endpoints et accélère les tests manuels.

L'activation de logs SQL Hibernate en environnement de développement simplifie le diagnostic et la compréhension des interactions ORM.

8.1 URLs utiles

- Swagger UI: <http://localhost:8080/swagger-ui/index.html>
- OpenAPI JSON: <http://localhost:8080/v3/api-docs>

9. Difficultés rencontrées et solutions

9.1 Fusion de branches

Un historique divergent entre branches a provoqué un merge conflict (README). La résolution a été effectuée en conservant la documentation la plus complète, puis validation sur main.

9.2 Configuration base de données

Le contexte local XAMPP sans mot de passe a imposé une configuration spécifique de datasource. Cette contrainte a été prise en charge dans application.properties et les manifestes Kubernetes.

9.3 Sécurité

Le paramétrage Security a été ajusté pour que UserDetailsService soit utilisé proprement, sans provider redondant.

10. Répartition du travail

La répartition doit être ajustée selon l'historique réel des commits du binôme. Exemple de répartition:

- Étudiant 1: modèle métier, sécurité, endpoints, Docker
- Étudiant 2: CI/CD, documentation, tests manuels, préparation rapport

Les commits signés sur GitHub servent de preuve objective de contribution.

10.1 Traçabilité

La stratégie adoptée privilégie des commits fréquents et ciblés afin de démontrer les étapes du développement et éviter les changements massifs peu lisibles.

11. Conclusion et perspectives

Le projet répond aux exigences principales du module DevOps: application fonctionnelle, workflow Git, conteneurisation, pipeline CI/CD, documentation claire et bonus Kubernetes.

Perspectives possibles: passage à JWT, tests d'intégration plus complets, monitoring (Prometheus/Grafana), déploiement cloud managé, et séparation des environnements dev/staging/prod.

Annexes recommandées

- Captures pipeline vert
- Captures Docker build/run
- Captures Swagger
- Captures kubectl get pods,svc
- Liens: dépôt GitHub, exécutions CI, image GHCR

Fin du rapport.